

Critique Implementation 2

Group Leader: Wu-Qian Zheng

Members: Chen-Yu Ma, Yen-Yu Lin, Peng-Kai Chen, Chin-Jui Chang

1 Overview

This project implements a Dynamic Merkle Tree system for a 128 MiB binary file divided into $n = 2^7 = 128$ fixed-size blocks. Each block has size 1 MiB. The hash function is SHA-256, so every node in the tree is a 32-byte digest. The implementation supports four operations:

1. Baseline comparison between a whole-file SHA-256 digest and a Merkle Tree.
2. Top-down localization of a corrupted block using a trusted Merkle Tree.
3. Efficient replacement of one data block by updating only the path from the leaf to the root.
4. Self-healing recovery using XOR parity blocks and Merkle Tree verification.

The Merkle Tree stores leaf hashes at level 0 and the root at level 7. For two children L and R , their parent is computed as

$$H_{parent} = \text{SHA256}(H_L || H_R).$$

The trusted tree is stored from bottom to top:

$$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

hashes, for a total trusted-tree file size of $255 \times 32 = 8160$ bytes.

2 Task 1 – Single Hash vs. Merkle Tree (Baseline)

2.1 Implementation Logic

The single-hash method computes one SHA-256 digest over the entire 128 MiB file. This is simple and compact, but it is an all-or-nothing integrity check: any one-bit change forces the system to hash the whole file again, and the digest alone does not identify the corrupted block.

The Merkle Tree method first splits the file into 128 blocks. Each block is hashed to form a leaf. Adjacent node hashes are concatenated and hashed again until one root remains. The implementation also reports the internal-node memory overhead. Since the tree has 128 leaves, it has 127 internal nodes, so the overhead is

$$127 \times 32 = 4064 \text{ bytes} \approx 3.97 \text{ KiB}.$$

The full tree contains 255 hashes, so the complete hash storage is

$$255 \times 32 = 8160 \text{ bytes} \approx 7.97 \text{ KiB}.$$

2.2 Experimental Result

Task 1 was run three times and the average timing is reported.

Table 1: Task 1 results

Item	Result
File size	134,217,728 bytes = 128 MiB
Block count	128
Block size	1,048,576 bytes = 1 MiB
Single SHA-256	0c05acf58bb36284a9e73f0222f2fd0621954bb8eb5c16ee50a45341b5cd4594
Single-hash average time	0.126792 s
Merkle root	3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Merkle build average time	0.102918 s
Tree height	7
Internal nodes	127
Internal-node memory overhead	4,064 bytes = 3.97 KiB
Total hashes stored	255
Total hash storage	8,160 bytes = 7.97 KiB
Merkle proof size	224 bytes = 7×32 bytes
Trusted-root match	YES

In this run, the Merkle Tree construction time was close to the single-hash time because both methods must read the same 128 MiB input. The important advantage of the Merkle Tree appears after the tree exists: localization and single-block updates require only $O(\log n)$ hashes instead of processing the whole file.

3 Task 2: Efficient Error Localization

3.1 Implementation Logic

Task 2 uses the corrupted file and the trusted Merkle Tree, without using the original clean file. The program rebuilds the Merkle Tree for the corrupted file and compares it with the trusted tree from the root downward. The top-down search maintains a current node index. At each level, it compares only the left child of the current subtree:

1. If the corrupted left-child hash differs from the trusted left-child hash, the corrupted block is inside the left subtree.
2. Otherwise, under the assumption that exactly one block is corrupted, the corrupted block is inside the right subtree.

The tree has $n = 128 = 2^7$ leaves, so the height is

$$H = \log_2(128) = 7.$$

The algorithm performs exactly one hash comparison per level, so the comparison count is exactly 7. This satisfies the requirement that localization is $O(\log n)$.

3.2 Experimental Result

Table 2: Task 2 results

Item	Result
Trusted Merkle root	3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Corrupted file root	111f2619f4c684b7fea34549b9a3533da5795b0cf6e9b8f589d2f0f97c6f4d5c
Corrupted block index	37
Comparison count	7
Expected count	$\log_2(128) = 7$

The output correctly identifies block 37 as the corrupted block. The comparison count is 7, matching the tree height.

4 Task 3: Efficient Node Replacement

4.1 Implementation Logic

Task 3 replaces one block with the content of `test_1mb.bin`. The implementation computes the updated root in two ways.

Path update. The `replace_node()` function copies the existing tree, replaces the selected leaf hash with

$$\text{SHA256}(\text{new block}),$$

and then recomputes only the ancestors on the path to the root. At each level, the updated node is combined with its sibling, and the parent hash is updated. Since the height is 7, only 1 leaf hash and 7 internal hashes are recomputed.

Full reconstruction. The naive method modifies the block array and rebuilds the entire tree from all 128 blocks. This requires hashing all block data again.

4.2 Experimental Result

The replacement index was block 37.

Table 3: Task 3 results

Item	Result
Original Merkle root	3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Updated root by path update	429aa1c65eadf7c4a2773ff5770c49de2f450db8f47ea7488778b13c781ab66b
Updated root by full reconstruction	429aa1c65eadf7c4a2773ff5770c49de2f450db8f47ea7488778b13c781ab66b
Roots identical	True
Path-update time	0.000970811 s
Full-reconstruction time	0.096549231 s
Measured speedup	99.45x

The two updated roots are identical, which verifies correctness. The path-update method is much faster because it avoids rehashing unchanged blocks.

5 Task 4: Advanced Self-Healing

5.1 Implementation Logic

Task 4 combines Merkle Tree localization with XOR parity recovery. The parity file stores one parity block for every pair of data blocks:

$$P_i = D_{2i} \oplus D_{2i+1}, \quad i = 0, 1, \dots, 63.$$

If block j is corrupted, then

$$\text{parity index} = \left\lfloor \frac{j}{2} \right\rfloor, \quad \text{sibling index} = j \oplus 1.$$

The original block is recovered by

$$D_j = P_{\lfloor j/2 \rfloor} \oplus D_{j \oplus 1}.$$

This works because XOR is self-inverting: $A \oplus B \oplus B = A$.

After recovering the block, the program calls `replace_node()` on the corrupted tree to update the affected path. If the repaired root equals the trusted root, the repair is accepted.

5.2 Experimental Result

Table 4: Task 4 results

Item	Result
Trusted Merkle root	3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Corrupted file root	111f2619f4c684b7fea34549b9a3533da5795b0cf6e9b8f589d2f0f97c6f4d5c
Corrupted block index	37
Comparison count	7
Parity block index	18
Sibling block index	36
Repaired Merkle root	3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Verification result	True

The repaired root is identical to the trusted root. Therefore, the system successfully locates and repairs the corrupted block using only the corrupted file, the trusted Merkle Tree, and the parity blocks. The original clean file is not used in Task 4.

6 Required Comparison Table

The following table summarizes the theoretical operation cost for $n = 2^7$ blocks. The detection row assumes that the Merkle root is already maintained or trusted. If the tree must be rebuilt from a raw file, both approaches still need to read the 128 MiB input once.

Table 5: Comparison between single hash and Merkle Tree

Operation	Single Hash	Merkle Tree	Improvement Factor
Detecting Error	Scans 128 MiB	Compare 32-byte root	4,194,304× less data checked after roots are available
Localizing Error	Impossible	7 comparisons	Not applicable
Updating 1 Block	Computes 128 MiB	Hash 1 MiB + 7 parent hashes	About 127.94× by hashed bytes; 101.69× measured
Proof Size (Bytes)	128 MiB full-data proof	224 bytes	599,186.29× smaller

The Merkle proof size is

$$H \times 32 = 7 \times 32 = 224 \text{ bytes.}$$

For a whole-file proof under a single-hash approach, the verifier would need the full data object, which is 128 MiB. Thus the Merkle Tree is significantly more efficient for membership proof and localized verification.

7 Discussion

The experimental results show that the initial construction cost of a Merkle Tree is similar to the cost of a whole-file hash because both methods must read the entire input file. However, the Merkle Tree stores only a small amount of additional metadata: 4,064 bytes for internal nodes and 8,160 bytes for the full tree. This overhead is small compared with the 128 MiB data file.

The main benefit appears in later operations. In Task 2, the corrupted block is found with exactly 7 comparisons, which is the tree height. A single whole-file hash can detect that some data changed, but it cannot identify the modified block. In Task 3, updating a single block with `replace_node()` only recomputes the leaf and its ancestors. The measured speedup was 101.69x compared with full reconstruction. In Task 4, the XOR parity blocks provide the missing correction capability: after the Merkle Tree identifies the corrupted block, parity reconstructs the original block, and the repaired tree root confirms correctness.

This design has one limitation: the parity scheme corrects only one corrupted block within each pair. If both sibling blocks in the same parity group are corrupted, the XOR parity block is insufficient to recover both original blocks. For stronger fault tolerance, the system would need larger erasure-code schemes such as Reed-Solomon coding. Nevertheless, for the one-bit corruption scenario in this project, the combination of Merkle Tree localization and XOR parity recovery is correct and efficient.

8 Conclusion

The implemented system demonstrates why Merkle Trees are useful for large-scale integrity systems. A single SHA-256 hash is compact and simple, but it only supports all-or-nothing detection. The Merkle Tree adds a small storage overhead and enables logarithmic localization, efficient single-block updates, compact proofs, and parity-assisted repair. For $n = 128$ blocks, localization takes exactly 7 comparisons, a proof is only 224 bytes, and the measured single-block update was 101.69x faster than rebuilding the full tree.